

ROBOT-ARM – Braț robotic controlat prin gesturile mâinii



UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA FACULTATEA DE AUTOMATICĂ ȘI
CALCULATOARE

Student: Dâncul Răzvan Adrian Grupa: 30221

Cuprins

1. LISTA DE ABREVIERI ȘI SIMBOLURI	3
2. INTRODUCERE	4
2.1 Justificarea abordării temei	4
2.2 Importanța și actualitatea temei	4
2.3 Comentarii sintetice privind realizarea	4
3. DESCRIEREA REALIZĂRII PRACTICE	5
3.1. Principiul funcțional.....	5
3.2 Schema electrică și componente	5
3.3. Probleme nerezolvate și motivarea lor.....	7
3.4. Observații privind rezultatele obținute	8
4. CONCLUZII	9
4.1. Utilitatea practică	9
4.2. Aspecte economice	9
5. BIBLIOGRAFIE	9
6. ANEXE	10
Anexa 1. Codul Sursă	11
A. Codul pentru mânășă senzorială	11
B. Codul pentru brațul robotic.....	13

1. LISTA DE ABREVIERI ȘI SIMBOLURI

- **ADC** – Analog to Digital Converter (Convertor Analog-Digital)
- **GND** – Ground (Masă/Pământare)
- **I2C** – Inter-Integrated Circuit (Protocol de comunicare serială)

- **IDE** – Integrated Development Environment (Mediu de dezvoltare integrat)
- **IMU** – Inertial Measurement Unit (Unitate de măsură inerțială)
- **PWM** – Pulse Width Modulation (Modulație în lățime de impuls)
- **RX/TX** – Receive / Transmit (Linii de comunicație serială)
- **UART** – Universal Asynchronous Receiver-Transmitter
- **VCC** – Voltage at the Common Collector (Tensiune de alimentare, +5V)
- **DOF** – Degrees of Freedom (Grade de libertate)

2. INTRODUCERE

2.1 Justificarea abordării temei

Prezentul proiect propune realizarea unui sistem robotic de tip "Master-Slave", compus dintr-un braț manipulator antropomorf (Slave) și o mână senzorială de control (Master). Am ales această abordare deoarece controlul prin mimarea mișcărilor naturale ale mâinii umane reprezintă cea mai intuitivă interfață om-mașină posibilă (HMI), eliminând curba de

învățare necesară operării joystick-urilor sau consolelor industriale complexe.

2.2 Importanța și actualitatea temei

În contextul industriei roboticii , teleoperarea joacă un rol crucial. Astfel de sisteme sunt vitale în medii periculoase pentru operatorul uman (manipularea deșeurilor toxice, dezamorsarea explozibililor sau lucrul în medii cu radiații), permițând intervenția precisă de la o distanță sigură. De asemenea, principiul stă la baza telemedicinei (chirurgia robotică la distanță) și a protezării bionice avansate.

2.3 Comentarii sintetice privind realizarea

Elementul de originalitate al proiectului constă în implementarea unui algoritm software de filtrare a datelor și prioritizare a mișcării. Deoarece senzorii analogici prezintă zgomot (jitter), iar mișcarea umană este rareori perfect izolată, am dezvoltat o logică de control care stabilizează semnalele și previne conflictele mecanice între articulațiile robotului, asigurând o mișcare fluidă. Totodată, un aspect inovator este reprezentat de sustenabilitatea proiectului, prin reciclarea anumitor materiale și integrarea acestora în ansamblul final.

3. DESCRIEREA REALIZĂRII PRACTICE

3.1. Principiul funcțional

Sistemul este divizat în două module independente care comunică wireless prin protocol Bluetooth:

1. **Modulul de Comandă (Mănușa):** Este echipat cu senzori de flexiune (rezistivi) plasați pe degete și două accelerometre/giroscoape MPU6050 (unul pe mână, unul pe antebraț). Microcontroller-ul culege datele, le convertește în valori digitale (0-1023 pentru flex, grade pentru IMU), le filtrează și construiește pachete de date seriale.

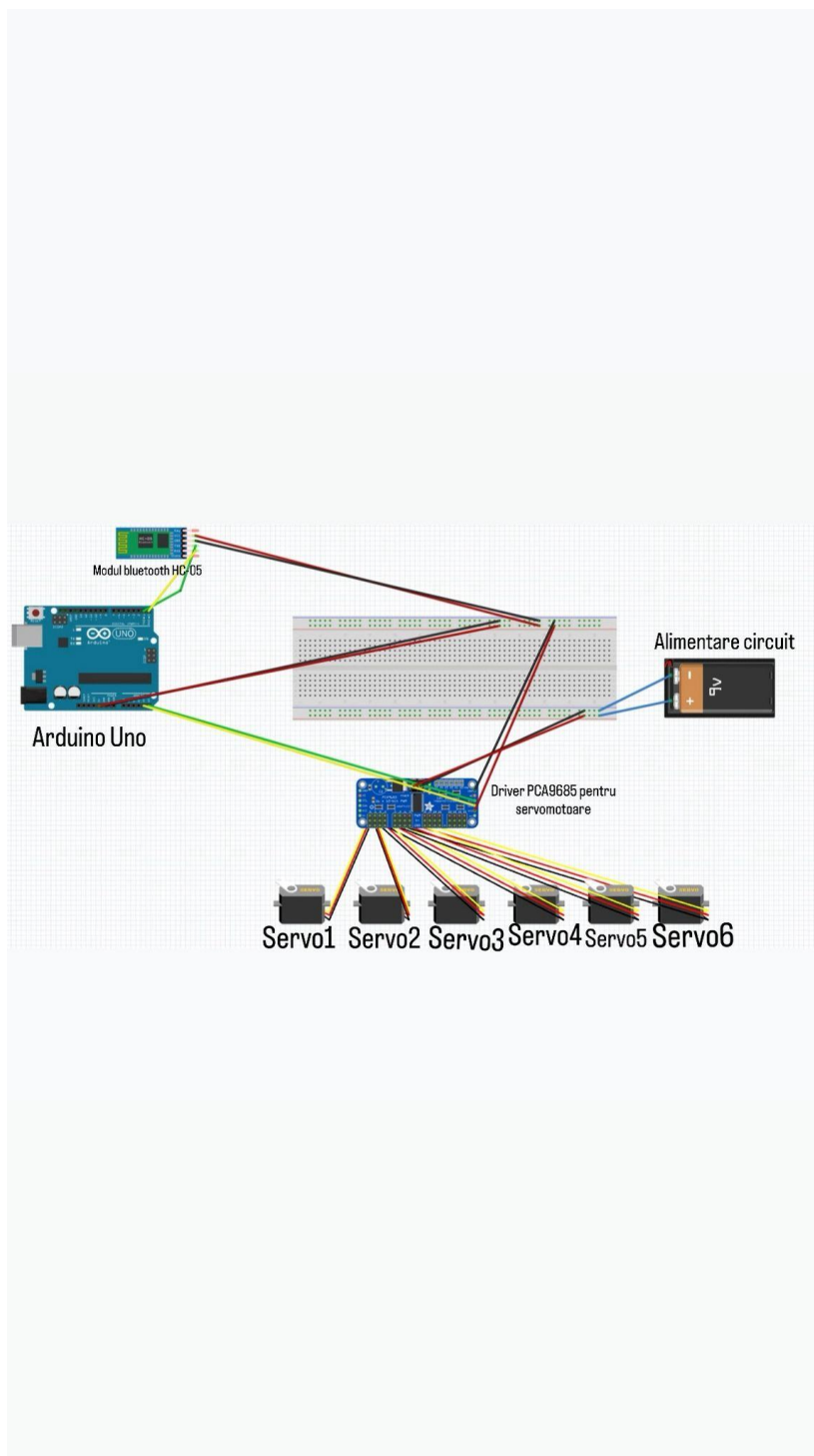
2. **Modulul de Execuție (Robotul):** Recepționează pachetul de date, decodează instrucțiunea (ex: caracterul 'F' pentru deschiderea cleștelui sau 'S' pentru rotirea bazei) și transmite semnale PWM către driverul de servomotoare PCA9685, care poziționează axele robotului.

3.2 Schema electrică și componente

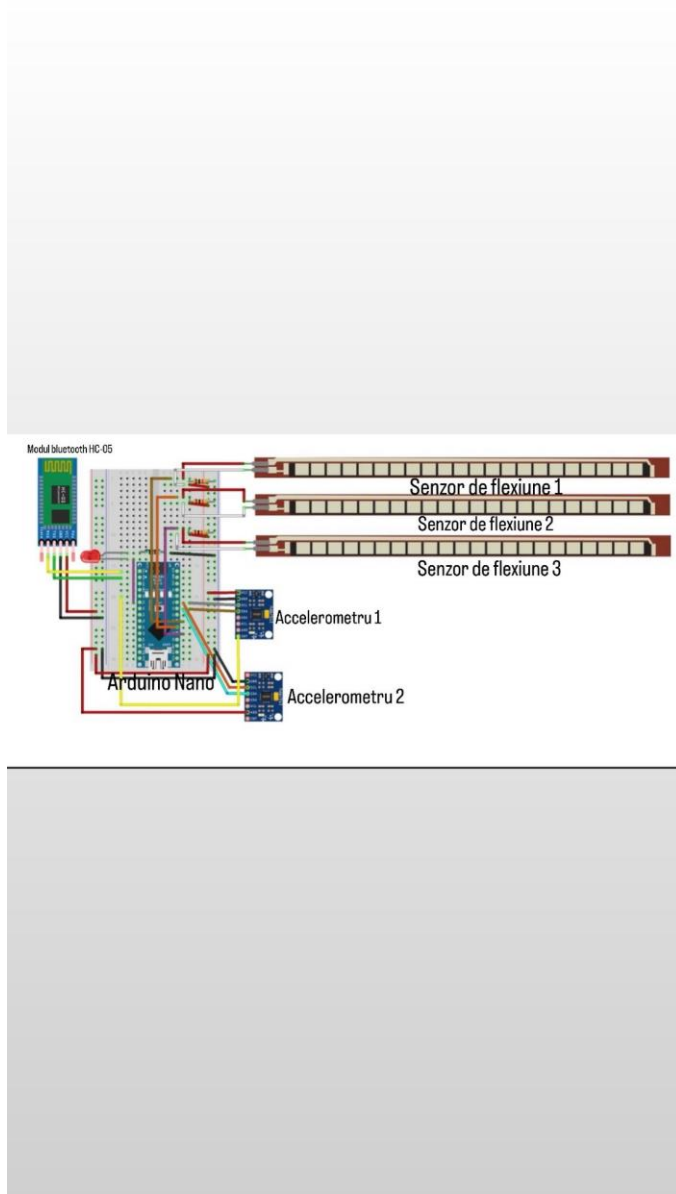
Sistemul integrează următoarele subsisteme electronice:

- **Senzorii de flexiune:** Conectați în configurație de divizor de tensiune cu rezistențe de pull-down ($10k\Omega$ - $30k\Omega$) la pinii analogici A0-A2.
- **Modulele IMU (MPU6050):** Conectate pe magistrala I2C (Pinii A4-SDA, A5-SCL). Pentru a utiliza două module simultan, adresele I2C au fost diferențiate (0x68 și 0x69) prin manipularea pinului AD0.
- **Comunicația:** Realizată cu module HC-05 conectate la pinii hardware UART (0 și 1) sau SoftwareSerial, în configurație încrucișată (TX Master -> RX Slave).
- **Acționarea:** Utilizarea unui driver PCA9685 permite controlul a până la 16 servomotoare prin I2C, degrevând procesorul Arduino de generarea semnalului PWM și permițând alimentarea separată a etajului de forță.

Schema electrică a robotului:



Schema electrică a mânușii:



3.3. Probleme nerezolvate și motivarea lor

Pe parcursul implementării și testării prototipului, au fost identificate două provocări tehnice majore, care au necesitat soluții software adaptate:

1. **Izolarea imperfectă a mișcărilor umane (Cuplaj biomecanic):** O dificultate majoră a proiectului o constituie necesitatea izolării riguroase a mișcărilor mâinii de cele ale brațului, pentru a asigura un control precis. Deși senzorii utilizează filtre parametrici de detecție, aceștia nu pot compensa micile variații cauzate de relaxarea involuntară a mușchilor sau neatenția operatorului. Prin urmare, este necesară menținerea unei poziții stabile, iar comenzile gestuale trebuie executate ferm pentru a evita interpretările eronate ale sistemului.

2. **Deconectarea modului Bluetooth prin cumularea comenzilor (Data Overflow):** În primele versiuni ale codului, rata de eșantionare a senzorilor era necontrolată, generând un flux continuu de caractere pe interfața serială. Această „rafală” de date ducea la cumularea rapidă a comenzilor nedorite în buffer-ul modului HC-05. Surplusul de informație depășea capacitatea de procesare a modului Slave, cauzând blocaje (freeze) sau deconectarea completă a legăturii radio.

a. Soluție parțială: Am redus rata de transmisie a comenzilor pentru a decongestiona comunicarea. Cu toate acestea, funcționarea optimă necesită în continuare ca utilizatorul să execute mișcările într-o manieră cât mai izolată și precisă.”

3.4. Observații privind rezultatele obținute

Prototipul final este funcțional și capabil să reproducă **6 grade de libertate (6 DOF)**: rotație bază, umăr, cot, încheietură (verticală și orizontală) și acționare clește. Timpul de latență este redus (sub 100ms) prin optimizarea ratei de transmisie a datelor, asigurând o operare fluidă în timp real.

4. CONCLUZII

4.1. Utilitatea practică

Proiectul demonstrează fezabilitatea unui sistem de teleprezență robotică cu cost redus. Acesta poate servi drept platformă didactică pentru studenți și bază pentru dezvoltarea protezelor bionice accesibile. De asemenea, sistemul are aplicabilitate în domeniul chimic (manipularea substanțelor toxice) și în telemedicină (intervenții medicale de la distanță).

4.2. Aspecte economice

Costul total al prototipului este estimat la aproximativ 465 RON, fiind semnificativ mai redus decât soluțiile industriale echivalente.

Componentă	Bucăți	Preț Unitar (Estimat)	Total (RON)
Placă dezvoltare compatibilă Arduino Uno	2	40 RON	80
Senzori Flexiune (sau materiale DIY)	3	30 RON	90
Accelerometru/Giroscoap MPU6050	2	15 RON	30
Modul Bluetooth HC-05	2	20 RON	40
Servomotoare (MG996R)	6	25 RON	150
Driver Servo PCA9685	1	25 RON	25
Structură Robot, Fire, Baterii	Set	50 RON	50
TOTAL GENERAL			465 RON

5. BIBLIOGRAFIE

- [1] Arduino LLC, *Arduino Language Reference*, Disponibil online: arduino.cc/reference/en
- [2] Bluetooth SIG, *Bluetooth Core Specification*, Versiunea 4.0.
- [3] InvenSense, *MPU-6050 Six-Axis MEMS MotionTracking™ Datasheet*, Rev. 3.4.
- [4] Texas Instruments, *PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller*, Datasheet.

6. ANEXE

Anexa 1. Codul Sursă

A. Codul pentru mănuașă senzorială

```
#include <Wire.h>

const int MPU_MANA = 0x68;

const int MPU_BRAT = 0x69;

const int PIN_INDEX = A0;

const int PIN_MIC = A1;

const int PIN_MARE = A2;

int INDEX_JOS = 180; int INDEX_SUS = 290;

int REST_JOS = 150; int REST_SUS = 230;

int PRAG_BRAT = 40;

int PRAG_ROTIRE = 45;

char lastRotire = ' ';

void setup() {

  Serial.begin(9600);

  Wire.begin();

  Wire.setWireTimeout(3000, true);

  Wire.beginTransmission(MPU_MANA); Wire.write(0x6B); Wire.write(0); Wire.endTransmission(true);

  Wire.beginTransmission(MPU_BRAT); Wire.write(0x6B); Wire.write(0); Wire.endTransmission(true);

}

void loop() {
```

```

int valIndex = analogRead(PIN_INDEX);

int valMic = analogRead(PIN_MIC);

int valMare = analogRead(PIN_MARE);


int yBrat = getVal(MPU_BRAT, 0x3D);

int zBrat = getVal(MPU_BRAT, 0x3F);

int xMana = getVal(MPU_MANA, 0x3B);


bool amTrimisCeva = false;


if (!amTrimisCeva) {

    if (valIndex < INDEX_JOS) { Serial.write('f'); amTrimisCeva = true; }

    else if (valIndex > INDEX_SUS) { Serial.write('F'); amTrimisCeva = true; }


    else if (valMare < REST_JOS) { Serial.write('P'); amTrimisCeva = true; }

    else if (valMare > REST_SUS) { Serial.write('p'); amTrimisCeva = true; }


    else if (valMic < REST_JOS) { Serial.write('G'); amTrimisCeva = true; }

    else if (valMic > REST_SUS) { Serial.write('U'); amTrimisCeva = true; }

}


if (!amTrimisCeva) {

    if (yBrat < -PRAG_BRAT) { Serial.write('S'); amTrimisCeva = true; }

    else if (yBrat > PRAG_BRAT) { Serial.write('O'); amTrimisCeva = true; }


    else if (zBrat < -PRAG_BRAT) { Serial.write('c'); amTrimisCeva = true; }

    else if (zBrat > PRAG_BRAT) { Serial.write('C'); amTrimisCeva = true; }

}


if (!amTrimisCeva) {

    char stareNoua = ' ';

```

```

if (xMana > PRAG_ROTIRE) stareNoua = 'H';

else          stareNoua = 'V';


if (stareNoua != lastRotire) {
    Serial.write(stareNoua);
    lastRotire = stareNoua;
    amTrimisCeva = true;
}
}

delay(120);
}

int getVal(int addr, int reg) {
    Wire.beginTransmission(addr); Wire.write(reg); Wire.endTransmission(false);
    Wire.requestFrom(addr, 2, true);
    if(Wire.available() < 2) return 0;
    int16_t val = Wire.read() << 8 | Wire.read();
    return map(val, -17000, 17000, -90, 90);
}

```

B. Codul pentru brațul robotic

```

#include <Wire.h>

#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

#define SERVOMIN 150
#define SERVOMAX 600

int currentPos[6] = {90, 90, 90, 90, 90, 90};

```

```

void setup() {

  Serial.begin(9600);

  pwm.begin();

  pwm.setPWMFreq(60);


  for(int i=0; i<6; i++) {

    moveServo(i, 90);

    delay(200);

  }

}


void loop() {

  if (Serial.available() > 0) {

    char state = Serial.read();

    int step = 5;


    if (state == 'f') clesteInchis();

    if (state == 'F') clesteDeschis();


    if (state == 'V') pozitieCioc();

    if (state == 'H') pozitiePalma();


    if (state == 'S') changePos(0, step);

    if (state == 'O') changePos(0, -step);


    if (state == 'C') changePos(1, step);

    if (state == 'c') changePos(1, -step);


    if (state == 'P') changePos(2, step);

    if (state == 'p') changePos(2, -step);

```

```

    if (state == 'U') changePos(3, step);

    if (state == 'G') changePos(3, -step);


    delay(10);
}
}

```

```

void clesteInchis() {

    int target = 145;


    if(currentPos[5] != target) {

        currentPos[5] = target;

        moveServo(5, target);

    }

}

```

```

void clesteDeschis() {

    int target = 65;


    if(currentPos[5] != target) {

        currentPos[5] = target;

        moveServo(5, target);

    }

}

```

```

void pozitieCioc() {

    int target = 50;

    if(currentPos[4] != target) {

        currentPos[4] = target;

        moveServo(4, target);

    }

}

```



```
}
```

```
void pozitiePalma() {
```

```
    int target = 180;
```

```
    if(currentPos[4] != target) {
```

```
        currentPos[4] = target;
```

```
        moveServo(4, target);
```

```
    }
```

```
}
```

```
void changePos(int servoIndex, int increment) {
```

```
    int newPos = currentPos[servoIndex] + increment;
```

```
    int limitaMinima = 0;
```

```
    int limitaMaxima = 180;
```

```
    if (servoIndex == 3) {
```

```
        limitaMinima = 40;
```

```
        limitaMaxima = 120;
```

```
    }
```

```
    if (newPos > limitaMaxima) newPos = limitaMaxima;
```

```
    if (newPos < limitaMinima) newPos = limitaMinima;
```

```
    if (newPos != currentPos[servoIndex]) {
```

```
        currentPos[servoIndex] = newPos;
```

```
        moveServo(servoIndex, newPos);
```

```
    }
```

```
}
```

```
void moveServo(int servoNum, int degree) {
```

```
int pulse = map(degree, 0, 180, SERVOMIN, SERVOMAX);  
pwm.setPWM(servoNum, 0, pulse);  
}
```